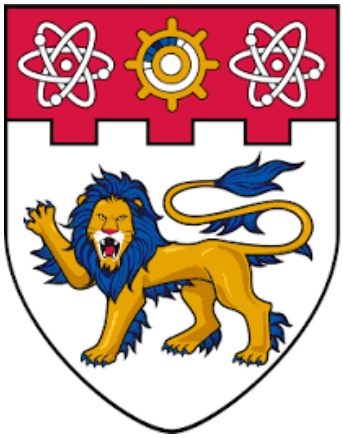


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4053 Blockchain Technology

AI Audit Report

Name	Matric Number

Content page

1. Initial Specification & System Scope **3**

1.1 Core requirements 3

1.2 Smart contract logic 4

1.2 1 Data Maintained by the Smart Contract 4

1.2.2 Domain Ownership and Resolution 6

1.2.3 Commit and Reveal Logic 6

1.3 UI features 8

1. Initial Specification & System Scope

Main functionalities: Domain registration, domain/address lookup (address-> [primary] domain and domain->address) interface

1.1 Core requirements

Feature	Requirements
Domain Availability for Bidding	● Smart contract rejects a domain that is already registered to an owner ● Smart contract rejects a domain that has an active auction
Initiate auction	● Contract allows an auction to be created only if domain availability has been fulfilled ● Contract records the auction parameters, including the domain identifier and phase deadlines ● Contract initialises the auction in the commit phase and transition to the reveal phase after the commit deadline
Commit Phase	● Bidders can only commit in an initiated auction during the commit phase ● In the frontend, bidders must specify intended bid amount in ETH (above fixed deposit) as well as the estimated conversion in SGD ● Bidders submit a cryptography commitment hash(domain,bid,salt) ● The bidder must transfer a fixed ETH deposit (eg 0.005) to the contract when submitting a commitment. ● The contract must record the deposit amount separately from the commitment hash, without storing the intended bid amount in plaintext. ● Each commitment must be associated with the bidder's address and the relevant auction. ● The contract must define consequences for bidders who fail to reveal their commits.
Reveal Phase	● The contract only accepts bid reveals after the commit deadline and before the reveal deadline ● The bidder must provide domain, bid amount and salt corresponding to their original commitment ● The contract must verify that the revealed information matches the stored commitment ● The contract must reject invalids reveals and prevent a commitment from being revealed more than once ● The contract must record each valid revealed bid and update the highest bidder when a higher valid bid is revealed ● The contract must determine the auction outcome after the reveal phase ends

Auction settlement and Domain Registration	● After the reveal phase, the contract must identify the highest valid bidder as the provisional winner ● The provisional winner must pay the full winning bid in ETH within a predefined settlement period ● The contract must register the domain to the winning bidder's address after full winning payment has been received ● The contact must store the winning bid and final auction status ● The winning bidder has to pay the full bid when auction settles, else apply a predefined penalty ● The contract must make deposits from unsuccessful bidders with valid reveals available for withdrawal after winner's payment has been received ● The contract must penalise bidders who fail to reveal their commits ● The contract must define how a winner's deposit is handled if they fail to make payment within settlement period and the domain is offered to the next highest valid bidder ● The contract must handle auctions with no valid revealed bids without registering the domain to an invalid winner ● The contract must prevent a settled auction from being settled again
Domain Resolution	● The contract must map each registered .ntu domain to its owner's address ● Any user must be able to query a domain, retrieve its resolved address and send ETH to the address ● The contract must allow a domain owner to designate one of their registered domains as their primary domain ● The contract must allow users to query an address and retrieve its designated primary domain, if it exists ● The contract must allow an owner to change their primary domain, and ensure only one primary domain is designated per address ● The contract must return an appropriate result for unregistered domain, no primary domain etc

FYI: **Fixed Deposit and Collateral**

The contract requires each bidder to submit a fixed ETH deposit when committing to a bid. The deposit serves as collateral to discourage failure to reveal a committed bid and to provide a predefined penalty for non-compliance. The fixed deposit also prevents deposit amounts from directly revealing individual bid amounts. Bid confidentiality is primarily provided by the cryptographic commitment scheme.

1.2 Smart contract logic

1.2 1 Data Maintained by the Smart Contract

1. Domain Registration and Resolution

- **Domain-to-owner mapping:** Associates each registered domain with its owner's Ethereum address.
- **Domain-to-resolved-address mapping:** Stores the Ethereum address to which each registered domain resolves. This may initially be set to the domain owner's address.
- **Owner-to-domains mapping:** Maintains a list of domains registered to each Ethereum address, allowing users to retrieve all domains they own.

2. Primary Domain (Bonus: Reverse Resolution)

- **Owner-to-primary-domain mapping:** Stores the primary .ntu domain designated by each Ethereum address for reverse resolution.

3. Auction-level Data

- **Domain identifier:** Identifies the domain being auctioned.
- **Auction status:** Indicates whether the auction is in the commit phase, reveal phase or settled.
- **Commit deadline:** Specifies the timestamp after which new bid commitments are rejected.
- **Reveal deadline:** Specifies the timestamp after which bid reveals are rejected.
- **Highest valid bid:** Tracks the highest valid bid revealed during the auction.
- **Highest bidder:** Stores the Ethereum address of the bidder currently holding the highest valid bid.
- **Settlement status:** Indicates whether the auction has been settled, preventing repeated settlement.

4. Bidder-level Data

- **Bidder address:** Identifies the Ethereum address submitting a commitment.
- **Commitment hash:** Stores the cryptographic hash of the domain identifier, bid amount, salt and bidder address.
- **ETH deposit:** Records the amount of ETH deposited by the bidder.
- **Reveal status:** Indicates whether the bidder has successfully revealed their bid.
- **Revealed bid amount:** Stores the bid amount after the commitment has been successfully verified.
- **Refund or penalty status:** Tracks whether the bidder's deposit has been refunded or forfeited according to the auction rules.

1.2.2 Domain Ownership and Resolution

Domain Registration Logic

1. When an auction is successfully settled, the contract identifies the winning bidder.
2. The contract assigns the domain to the winning bidder's Ethereum address.
3. The contract records the domain-owner association and adds the domain to the winner's list of registered domains.
4. The contract initialises the domain's resolved address to the winner's Ethereum address.

Primary-Domain Selection Logic

1. A user requests to designate a domain as their primary domain through the frontend.
2. The contract verifies that the requested domain is registered and owned by the requesting address.
3. If the ownership check passes, the contract updates the user's primary-domain mapping to the selected domain.
4. If the user already has a primary domain, the new selection replaces the previous selection.
5. If the ownership check fails, the transaction is rejected.

Resolution Logic

- Forward resolution: given a domain, the contract resolves the associated Ethereum address and allows the user to send ETH using the domain.
- Reverse resolution (bonus): given an Ethereum address, the contract returns its designated primary domain, if one exists.
- If no matching domain or primary-domain designation exists, the contract returns an appropriate empty or unregistered result.

1.2.3 Commit and Reveal Logic

Auction Initiation

1. The contract checks that the requested domain is valid, unregistered and not subject to an active auction.
2. If all checks pass, the contract creates an auction record containing the domain identifier, commit deadline, reveal deadline and auction status.
3. The auction enters the commit phase.

Commit Phase

1. The bidder specifies a bid amount and generates a random salt through the frontend.

2. The frontend calculates a cryptographic commitment hash using the domain, bid amount, salt and bidder address.
3. The bidder submits the commitment hash and required ETH deposit to the contract.
4. The contract verifies that the auction is in the commit phase and records the commitment against the bidder and auction.
5. The contract rejects commitments submitted after the commit deadline.

Reveal Phase

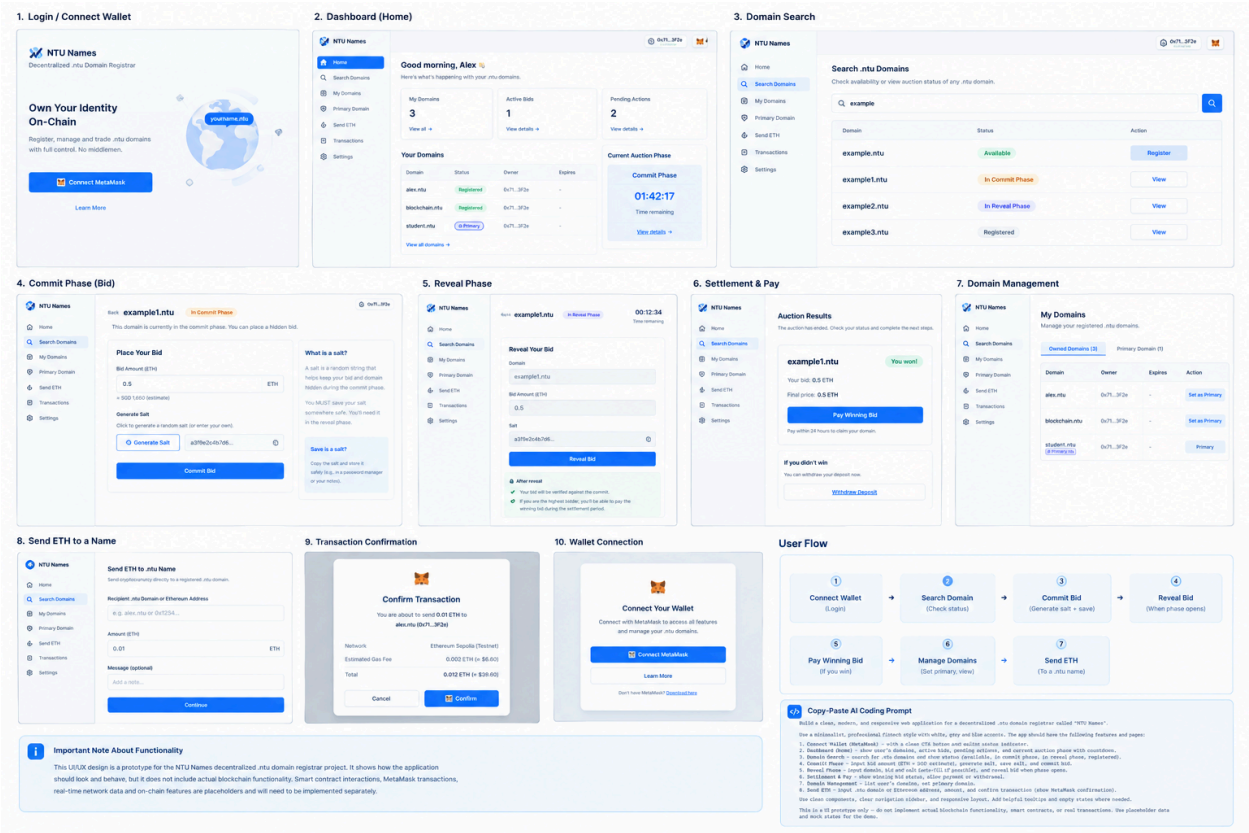
1. After the commit deadline, the auction enters the reveal phase.
2. The bidder submits the domain, bid amount and salt associated with their commitment.
3. The contract recalculates the commitment hash and compares it with the previously stored hash.
4. If the hash matches and the bid satisfies the deposit requirements, the contract records the valid bid.
5. The contract updates the highest valid bid when a higher bid is revealed.
6. Invalid reveals and duplicate reveals are rejected.

Auction Settlement

1. After the reveal deadline, the contract must determine the highest valid revealed bid and identify the provisional winner.
2. The provisional winner must pay the full winning bid in ETH within a predefined settlement period.
3. The contract must register the domain to the winning bidder only after the full winning payment has been received.
4. Upon successful settlement, the contract must record the winning bid, transfer or account for the winning payment according to the auction rules, refund the winner's collateral deposit, and make eligible losing bidders' collateral deposits available for withdrawal.
5. If the provisional winner fails to pay within the settlement period, the contract must apply the predefined collateral penalty and follow the specified fallback procedure, such as offering the domain to the next-highest valid bidder or restarting the auction.
6. The contract must penalise bidders who fail to reveal their commitments according to the predefined penalty rules.
7. If no valid bids are revealed, the contract must mark the auction as settled without a winner and leave the domain unregistered.
8. The contract must prevent an auction from being settled more than once.

1.3 UI features

- **Search:** a domain input showing status (available, in commit phase, in reveal phase, registered with owner and resolved address).
- **Commit:** bid amount in ETH, an SGD estimate, a generate-salt step, a “Commit” button, and a prompt to save the salt.
- **Reveal:** domain, bid and salt inputs (ideally auto-filled from saved data) and a “Reveal” button.
- **Settle and pay:** a “Pay winning bid” button for the winner within the settlement period, and a “Finalise auction” button.
- **Withdraw:** a “Withdraw deposit” button for losing bidders.
- **Primary domain:** a list of the user’s domains with a “Set as primary” action.
- **Send ETH to a name:** a `.ntu` name input that resolves to an address, then a send button.
- **Wallet and phase info:** a connect-wallet button, plus a countdown or phase indicator for the current auction.



Wow javier generated this mockup and its basically everything we need lol (just for our reff)

[UI Descriptions]

Overview

Slides 1-3: extra information
Slides 4-6: Connecting MetaMask
Slide 7: Home
Slide 8: Search domains
Slide 9: My domains
Slides 10-12: Send ETH
Slide 13: Transaction history
Slides 14-17: Commit bid
Slides 18-20: Reveal bid and payment
Slides 18, 21-22: Reveal bid and withdrawal

Specific buttons

Slide 4 (First page of website)

- [Connect MetaMask] → Slide 5

Slide 5 (Select MetaMask account to connect to our website)

- [Cancel] → Back to Slide 4
- [Next] → Slide 6

Slide 6 (Confirmation)

- [Back] → Back to Slide 5
- [Connect] → Slide 7

Slide 7 (Home / Dashboard)

- [Commit bid] → Slide 14
- [Reveal bid] → Slide 18
- [Settle payment] → Slide 19 or 21

Slide 8 (Search domains)

- [Commit bid] → Slide 14
- [Reveal bid] → Slide 18
- [Settle payment] → Slide 19 or 21
- [View details] → Slide 9 (My domains)
- [Search] → Returns a list of closest results below

Slide 9 (My domains)

- [Reveal bid] → Slide 18
- [Settle payment] → Slide 19 or 21
- [Set primary] → Changes the status of the domain from “Registered” to “Primary,” and the initial “Primary” one is changed to “Registered”

Slide 10 (Send ETH)

- User fills out the fields
- [Proceed] → Slide 11

Slide 11

- [Back] → Slide 10
- [Continue to MetaMask] → Slide 12

Slide 12

- [Back to Home] → Slide 7 (Home)

Slide 13 (Transaction history)

- No buttons

Slide 14 (Commit bid)

- User fills out bid amount, and SGD estimate appears
- [Generate random salt] → Slide 15

Slide 15 (Commit bid)

- Salt appears, user has to tick checkbox to commit bid
- [tick checkbox] → Slide 16

Slide 16 (Commit bid)

- Now user can commit
- [Commit bid] → Slide 17

Slide 17 (Commit bid confirmation)

- [Back to Home] → Slide 7 (Home)

Slide 18 (Reveal bid)

- User keys in saved salt

- [Reveal bid]
  Slide 19 (If win)
    - [Pay winning bid] → Slide 20

  Slide 21 (If lose)
    - [Withdraw deposit] → Slide 22

Slide 20
- [Back to Home] → Slide 7 (Home)

Slide 22
- [Back to Home] → Slide 7 (Home)

FYI sections to include

1. Initial Specification & System Scope: A description of the core requirements, smart contract logic, and UI features you intended to build before engaging AI tools.

2. Development Methodology & AI Tooling Infrastructure: A detailed breakdown of your development workflow and the toolchain used, including:

• The AI coding assistants, IDE plugins, or agentic tools utilized (e.g., Claude Code, Kiro, Cursor, GitHub Copilot).

• Any custom AI agents, automated scripts, specification files (spec.md, design blueprints), or LLM workflows constructed by your team to aid in contract generation, front-end binding, or automated testing.

• How AI was integrated into your team's division of labor and decision-making process.

3. Specification Engineering & Prompt Logs: How requirements were communicated to AI tools:

• Formal specification files written for AI coding tools/agents.

• Representative prompt chains, iterations, and context windows provided during generation.

4. AI-Generated Architecture: An overview of the structural design, contract interfaces, and state management initially proposed by the AI tools/agents.

5. Problems Discovered & Vulnerability Analysis: A comprehensive inventory of bugs, security vulnerabilities (e.g., reentrancy, access control flaws, integer/math issues, unhandled gas limits), inefficiencies, or logic errors found in the AI's generated output.

6. Validation Process & Testing Strategy: The methods (e.g., unit testing suites like Foundry/Hardhat, static analysis tools like Slither, manual code review, testnet simulation)

used to test and validate the AI-assisted codebase.
7. Improvements & Refinements: Functional additions, architectural modifications, and
code optimizations made beyond the base AI output.
8. Lessons Learned: Your team's critical evaluation of the AI tools and methodology used—
highlighting where AI excelled, where it failed, and recommendations for future AI-driven
dApp development.

2.2 Major Issue Breakdown Format

For every major issue or security vulnerability discovered in the AI-generated architecture or
codebase, students must provide a detailed breakdown covering:
1. Why AI made the mistake: (e.g., hallucinated deprecated Solidity syntax, misunderstood
state variable scoping across transactions, lacked context on reentrancy attack vectors,
misconfigured front-end web3 provider states).
2. Why it is problematic: The potential real-world impact (e.g., risk of drainable funds,
stuck/bricked tokens, unauthorized admin privilege escalation, inaccurate price calcula
tion).
3. How they fixed it: The specific code refactoring, manual adjustments, or re-prompting/spec
updates implemented to resolve the flaw.
4. Evidence the fix works: Proof of resolution (e.g., passing unit test execution logs, Slither
security check outputs, transaction hashes on testnets, state validation screenshots)